

A Configurable Street-Level Vision Node for Curio: Bringing Pre-Trained CV into Urban Visual Analytics Workflows

Lakshmi Sravya Rachakonda Laxmi Sai Maneesh Reddy Jupalle
University of Illinois at Chicago

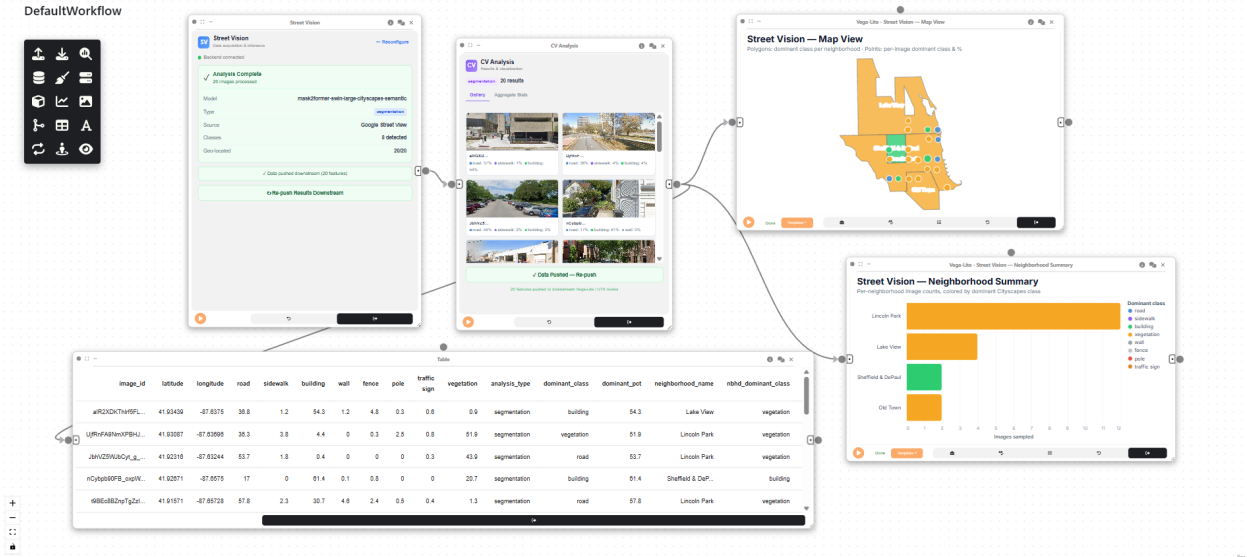


Fig. 1: The Street-Level Vision Analytics extension on the Curio canvas. A *Street Vision* node (place search, model selection, inference) feeds a *CV Analysis* node, which emits a GeoJSON FeatureCollection enriched with Chicago neighborhood polygons. Three downstream consumers read different projections of the same payload: a Vega-Lite Map View coloured by modal Cityscapes class, a Vega-Lite Neighborhood Summary bar chart of per-neighborhood image counts, and a Table node exposing the per-image feature row.

Abstract—Urban analysts increasingly want to derive block-level signals (greenery, sidewalk coverage, vehicle composition, accessibility infrastructure) from street-level imagery, but the existing path requires custom Python scripts, model loading boilerplate, and ad-hoc spatial joins that put computer vision (CV) out of reach for non-programmers. We extend Curio, a node-based urban visual analytics platform, with a two-node CV pipeline: a *Street Vision* node that performs place-name search, Google Street View sampling, and pre-trained model inference, and a *CV Analysis* node that handles result inspection, per-class metrics, neighborhood enrichment, and GeoJSON/DataFrame export to downstream Vega-Lite and UTK nodes. Users select any HuggingFace segmentation or detection checkpoint via the in-node search; our demo uses Mask2Former-Swin-Large on Cityscapes for segmentation and supports YOLOv8 for detection. We demonstrate the system on a Chicago greenery assessment and a vehicle-counting case study, document seven newly enabled analytical tasks, and report representative inference latency on CPU. The paper closes with limitations around demo-mode batch caps, the fixed Cityscapes taxonomy, and the spatial sparsity of Street View metadata coverage.

Index Terms—Urban visual analytics, dataflow systems, semantic segmentation, street-level imagery, Curio.

1 INTRODUCTION

Cities now publish or stream petabyte-scale street-level imagery (Google Street View, Mapillary, municipal LiDAR rigs), and a decade of computer-vision research has shown that pre-trained semantic segmentation and object detection models can extract policy-relevant signals from those images: tree canopy [5], sidewalk condition, vehicle composition [3], and pedestrian infrastructure. The catch is workflow friction. An urban planner asking “which Lincoln Park blocks have less than 10% sidewalk?” has to learn HuggingFace’s API [9], write image-fetching code against the Street View Static API, manage a CUDA environment, run inference, then stitch the results into a GIS workflow alongside census tracts. The CV is the easy part; the plumbing is what blocks adoption.

Node-based visual analytics platforms such as Curio [7] and KN-IME [1] were designed to remove exactly this kind of plumbing for

tabular and geospatial workflows, but none of them ship a node that runs a pre-trained CV model on live street imagery. We close that gap. Our contribution is a *two-node* extension to Curio that handles model selection, place-name search, Street View sampling, inference, neighbourhood enrichment, and export to Vega-Lite and UTK, all behind a configuration UI that exposes no Python (Figure 1). Concretely we contribute (1) a **Street Vision** node that wraps HuggingFace model search and Google Street View sampling behind a place-name interface, (2) a **CV Analysis** node that performs server-side spatial joins against Chicago neighborhood polygons and emits both GeoJSON and DataFrame outputs, and (3) a Curio-native Vega-Lite *Map View* template that renders neighborhoods coloured by their modal Cityscapes class. We evaluate via a seven-task analytical inventory and two case studies (Chicago greenery; vehicle counting in The Loop).

2 RELATED WORK

Urban visual analytics platforms. Curio [7] introduced a dataflow abstraction for urban analytics that lets analysts wire together data acquisition, transformation, and visualization nodes; the underlying Urban Toolkit (UTK) [6] provides a grammar for thematic urban maps. These platforms benefit from interactive, multi-view design but historically treat CV outputs as pre-computed inputs rather than a node in the live graph. Our work integrates the inference step itself into the graph.

Street-level imagery analysis. Prior work uses street-level imagery to infer block- or neighborhood-scale attributes: Treepedia [5] estimated street-level greenery from Google Street View at city scale, and Gebru et al. [3] used vehicle make/model recognition over 50M Street View images to infer demographics. These studies are typically one-off pipelines in research code; we package the workflow as reusable nodes so non-programmers can re-run the same kind of analysis on new bounding boxes or with substituted models.

Pre-trained CV for cities. SegFormer [10] achieves state-of-the-art accuracy on the Cityscapes 19-class urban benchmark [2], and YOLOv8 [4] provides fast object detection on commodity CPUs. The HuggingFace Hub [9] aggregates thousands of such checkpoints; we expose the Hub’s search API directly inside the Street Vision node so users can swap models without code changes. We treat the transferability of Cityscapes-trained models to U.S. street imagery as a working assumption rather than a contribution.

No-code visual analytics. Vega-Lite [8], KNIME [1], and Tableau define the modern expectation that data analysts work without writing imperative code. Our extension follows that tradition: every parameter the user might want to change (model, location, target classes, filtering operator) is a UI control, not a Python edit. The heavy work (CV inference) happens server-side and only the resulting GeoJSON is shipped to the client.

3 DATA AND TASKS

Imagery. We consume images from the Google Street View Static API and use the companion Metadata API for free coverage probing before any paid request. A user-supplied place name (e.g. “Lincoln Park Chicago”) is geocoded by OpenStreetMap Nominatim into a bounding box; the box is sampled on a regular grid; each sample is checked against the Metadata API; covered samples are then fetched as 640×480 images at 90° field of view. The sampling density and per-request cap are configurable on the Street Vision node.

Spatial reference. For Chicago we ship a curated GeoJSON of 98 neighborhood polygons, indexed in memory with a Shapely STRtree so that point-in-polygon joins for hundreds of inference results complete in well under a second. Each result feature is tagged with its neighborhood name, and per-neighborhood roll-ups (modal class, mean dominance percentage, image count) are computed server-side and exposed at /api/data/basemap/enrich_with_neighborhoods.

Models. The system has no hardcoded default model. The user picks any HuggingFace semantic segmentation or detection checkpoint via the in-node search, with the option of auto-picking the top result by download count for a given query. The backend’s loader is architecture-agnostic: it tries `AutoModelForSemanticSegmentation`, then `AutoModelForUniversalSegmentation`, then `AutoModelForInstanceSegmentation`, so SegFormer, BEiT, and DPT load through the first path while MaskFormer, Mask2Former, and OneFormer load through the universal path. Our case study below uses facebook/mask2former-swin-large-cityscapes-semantic, the top auto-pick result for the query “cityscapes” on the day of the demo, predicting the standard 19-class Cityscapes taxonomy. For object detection we use ultralytics/yolov8n for the vehicle-counting case study.

Tasks. Table 1 summarizes seven analytical tasks the node enables; full descriptions, target user roles, and “how it was done before” baselines are documented in the project repository. The tasks span environmental equity (greenery, seasonal vegetation), pedestrian infrastructure (sidewalk coverage, ramps, street furniture), and verification workflows (manual CV inspection). The two case studies in section 5 cover tasks 1 and (in spirit) 2.

Table 1: Seven analytical tasks newly enabled by the node, by representative model class and primary metric. Time-savings estimates compare the node’s interactive workflow against the manual or scripted baselines documented in the project’s task inventory.

#	Task	Model class	Primary metric
1	Compare greenery across neighborhoods	semantic seg.	vegetation ratio
2	Identify blocks with no street furniture	detection	furniture count
3	Verify a CV result against the photo	any	visual + bar
4	Find blocks with poor sidewalk coverage	semantic seg.	sidewalk / road
5	Audit accessibility ramp presence	detection	ramp count
6	Track seasonal vegetation change	semantic seg.	seasonal delta
7	Correlate CV metrics with census tracts	either	cross-dataset

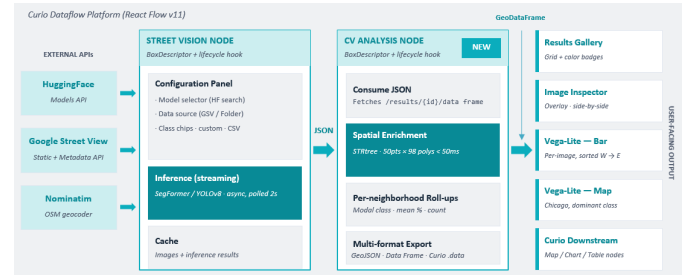


Fig. 2: System architecture. Place name → Nominatim → bounding box → Street View Metadata API (coverage check) → Street View Static API (image fetch) → HuggingFace model inference → CV Analysis (Shapely STRtree enrichment) → GeoJSON / DataFrame outputs → downstream Vega-Lite, Table, and UTK nodes.

4 SYSTEM DESIGN

Two-node split. An early prototype crammed all of model loading, fetching, inference, and visualization into one Curio node; this collapsed Curio’s dataflow story into a black box and was rejected after instructor feedback. The shipped design splits computation across two nodes (Figure 2). The *Street Vision* node owns model selection, place search, sampling, and inference, and emits a structured JSON payload. The *CV Analysis* node consumes that payload, runs the spatial-enrichment join, exposes the gallery / inspector / filter UI, and publishes a GeoJSON FeatureCollection on its output port (a column-oriented DataFrame projection of the same data is reachable via a separate REST endpoint for headless consumers). Downstream Vega-Lite and UTK nodes can be wired to that output without knowing anything about the upstream model.

Configuration UI (Figure 3). The Street Vision sidebar is a three-step wizard: (1) model search, (2) data source, and (3) target classes. Each step has sensible defaults so a first-time user can run an analysis in under a minute. Model search hits the live HuggingFace API and is filtered by task type (segmentation or detection); class chips come pre-populated from the Cityscapes 19-class CSV but accept free-text input and CSV uploads for custom taxonomies. The data-source step supports both Street View and a local-folder fallback for offline use.

Inference pipeline. On submit, the backend (a) loads the selected model into a process-wide cache so repeated runs amortize the load cost, (b) batches images through the model on CPU, (c) for segmentation, computes per-pixel class predictions and writes both a logits-derived overlay PNG and a per-class pixel-ratio dictionary, and (d) returns a job id that the frontend polls for status and final results. As a representative reference point, our benchmark runs of SegFormer-B2 at the model’s native 1024×1024 input averaged 0.30 s per image on a 4-core CPU. Mask2Former-Swin-Large is a substantially larger checkpoint and was used only for the visual case study, not for latency benchmarking.

Spatial enrichment. The CV Analysis node sends the inference

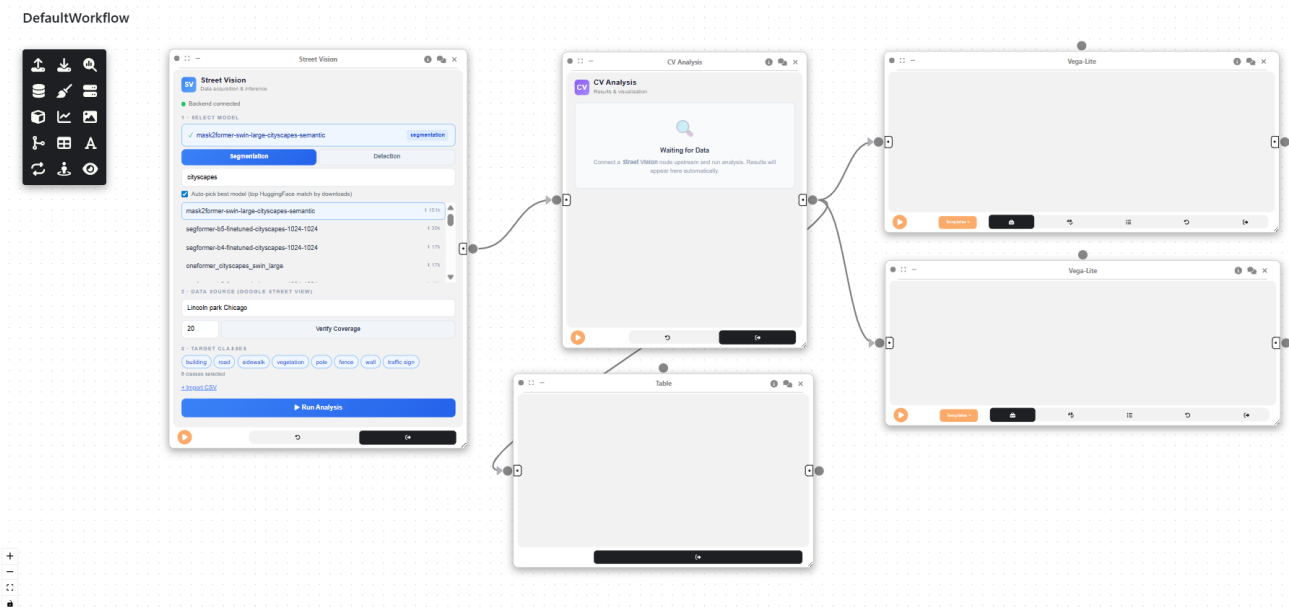


Fig. 3: The Street Vision node’s three-step configuration wizard, ready to run. Step 1 lists live HuggingFace results sorted by downloads; Step 2 takes a place name and a sample limit; Step 3 exposes Cityscapes class chips plus a CSV upload affordance. The downstream CV Analysis node sits idle with “Waiting for Data” until the user hits *Run Analysis*.

results to `/api/data/basemap/enrich_with_neighborhoods`, which (1) constructs a Shapely Point per result in the basemap’s native WGS84, (2) does a single STRtree point-in-polygon query per point, (3) groups by neighborhood and computes per-neighborhood modal class, mean dominance percentage, and image count, and (4) returns both the per-point tags and the per-neighborhood aggregates in one response. Doing this server-side keeps the Vega-Lite and UTK clients stateless: they only see a GeoJSON ready to render.

Map View template. A built-in Vega-Lite spec, selectable from the Vega-Lite node’s template menu, paints each searched neighborhood with the colour of its modal class, labels it with the neighborhood name, and overlays per-image points sized by dominance percentage. The same colour palette is shared across the segmentation overlay, the per-class bar chart in the inspector, and the Map View, so a building (green) on the canvas is the same green in the chart and on the map.

Implementation. Backend: Python 3.10, FastAPI, Transformers, Ultralytics, GeoPandas, Shapely. Frontend: React 18 with TypeScript and Vite. Curio integration: a forked Curio repo with two new `BoxDescriptor` entries and matching React lifecycle hooks; the diff is shipped as a single `curio-fork.patch`. Total project outside the Curio fork: ~ 1.8 k lines of TypeScript and ~ 1.6 k lines of Python.

5 EVALUATION

Case study 1: Chicago greenery (Figure 4). We ran Mask2Former-Swin-Large fine-tuned on Cityscapes against Street View samples from a Lincoln Park bounding box, with target classes {vegetation, road, building, sidewalk, sky}. The Map View renders Lincoln Park (and any other searched neighborhoods) coloured by its modal class; the Neighborhood Summary bar chart, sorted by image count, makes the gradient between residential blocks (more vegetation) and arterial corridors (more road) immediately visible. Compound filters (e.g. `vegetation < 0.10`) surface candidate blocks for tree-planting prioritization without any code. The same workflow re-runs identically on Hyde Park or The Loop by typing a new place name.

Case study 2: Vehicle counting in The Loop. Swapping the model to `ultralytics/yolov8n` and the class chips to {car, truck, bus, motorcycle, bicycle}, the same node returns object-count results instead of pixel ratios; the Map View template adapts because it reads from the same per-neighborhood roll-up endpoint. This is the configuration-generality claim made concrete: zero code changes between the two

case studies.

Performance. Per-image inference latency depends on the chosen checkpoint. SegFormer-B2 on a 4-core CPU runs at ~ 0.30 s per image, yielding ~ 3.3 images/s end-to-end including HTTP and post-processing. Mask2Former-Swin-Large is roughly an order of magnitude larger and was not benchmarked formally; the case studies use the standard demo batch of 20 images so a single run completes in well under a minute even on the larger checkpoint. The backend caps requests at 200 images per call, primarily to keep Google Street View API quota predictable for an interactive workflow; lifting the cap is a one-line change in `inference_service.py` for offline runs.

Multi-node dataflow. The teaser figure (Figure 1) shows three downstream consumers wired to CV Analysis: a Vega-Lite Map View of Chicago neighborhoods coloured by their modal Cityscapes class, a Vega-Lite Neighborhood Summary bar chart of per-neighborhood image counts coloured by dominant class, and a Table node exposing the per-image feature row (latitude, longitude, per-class ratios, dominant class, neighborhood name). UTK consumption is supported via the same GeoJSON port but is not pictured. This is the Curio value proposition we set out to honour: not a monolithic CV widget, but a CV stage that participates in the dataflow graph.

6 DISCUSSION

What worked. Splitting the pipeline across two nodes was the right call. It forced a clean serialization boundary between inference and visualization, and the GeoJSON export turned out to slot directly into Curio’s existing Vega-Lite and UTK consumers with no special-casing. Pre-trained models from HuggingFace were almost always good enough; we never needed to fine-tune. Server-side spatial joins via STRtree kept the client thin and made the Map View template trivially fast.

What was harder than expected. Coverage estimation against the Street View Metadata API turned out to be the dominant source of UX friction, not inference latency. Many Chicago bounding boxes are unevenly covered, and a user who picks a name like “Garfield Park” and gets back four images out of fifty assumes the model is broken. We surface a coverage percentage in the UI to set expectations but cannot fix the underlying sparsity. The fixed Cityscapes taxonomy is also a real limit: street furniture, ramps, and seasonal phenology all sit outside the 19 classes and would need either a different checkpoint (e.g. open-vocabulary models like CLIPSeg or SAM) or fine-tuning.

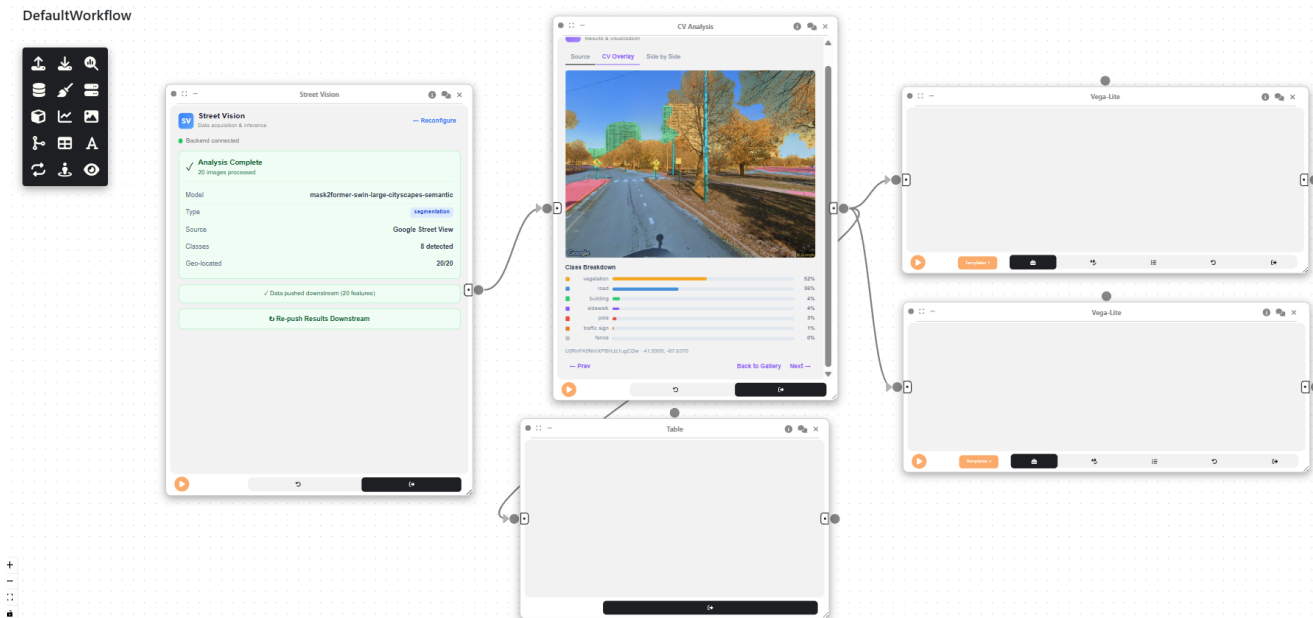


Fig. 4: The CV Analysis node's Image Inspector inside the Curio canvas. A Lincoln Park Street View image is shown with the Mask2Former-Swin-Large segmentation overlay, alongside a per-class breakdown bar chart that uses the same palette as the Map View.

Limitations. (1) The 200-image per-request cap is a quota-protection measure, not a fundamental limit; large-area sweeps still need to be issued as multiple calls and reassembled downstream. (2) Street View has known temporal sparsity in residential blocks; metric stability across runs depends on which panoramas were captured most recently. (3) Inference is CPU-only; a GPU-backed deployment is straightforward but out of scope for the deliverable. (4) The neighborhood enrichment dataset is Chicago-only; extending to other cities is a configuration change but currently undocumented.

7 CONCLUSION AND FUTURE WORK

We presented a two-node Curio extension that brings pre-trained street-level CV into a no-code dataflow workflow, demonstrated on Chicago greenery and vehicle counting case studies, with seven analytical tasks documented. The natural next steps are (1) GPU-backed inference and a real scaling study, (2) Integrating open-vocabulary models so the target classes are not bound to Cityscapes, (3) Shipping neighborhood basemaps for additional U.S. cities, (4) Adding a custom-model upload affordance, and (5) A user study to validate the time-savings claims in Table 1.

Code availability. Source, demo screenshots, and reproduction scripts are at <https://github.com/ManeeshJupalle/Street-Level-Vision-Analytics-Node-for-Curio>.

ACKNOWLEDGMENTS

We thank the CS 524 instructor and teaching staff at the University of Illinois Chicago for feedback throughout the semester, in particular the recommendation that drove the two-node split. The Curio project authors made integration straightforward by keeping the BoxDescriptor extension surface stable.

REFERENCES

- [1] M. R. Berthold, N. Cebon, F. Dill, T. R. Gabriel, T. Kötter, T. Meinl et al. KNIME – the Konstanz information miner: Version 2.0 and beyond. *ACM SIGKDD Explorations Newsletter*, 11(1):26–31, 2009. doi: 10.1145/1656274.1656280 1, 2
- [2] M. Cordts, M. Omran, S. Ramos, T. Rehfeld, M. Enzweiler, R. Benenson et al. The Cityscapes dataset for semantic urban scene understanding. In *Proc. CVPR*, pp. 3213–3223, 2016. doi: 10.1109/CVPR.2016.350 2
- [3] T. Gebru, J. Krause, Y. Wang, D. Chen, J. Deng, E. L. Aiden et al. Using deep learning and Google Street View to estimate the demographic makeup of neighborhoods across the United States. *Proc. National Academy of Sciences*, 114(50):13108–13113, 2017. doi: 10.1073/pnas.1700035114 1, 2
- [4] G. Jocher, A. Chaurasia, and J. Qiu. Ultralytics YOLOv8. <https://github.com/ultralytics/ultralytics>, 2023. 2
- [5] X. Li, C. Zhang, W. Li, R. Ricard, Q. Meng, and W. Zhang. Assessing street-level urban greenery using Google Street View and a modified green view index. *Urban Forestry & Urban Greening*, 14(3):675–685, 2015. doi: 10.1016/j.ufug.2015.06.006 1, 2
- [6] G. Moreira, M. Hosseini, M. N. Alam Nipu, M. Lage, N. Ferreira, and F. Miranda. The Urban Toolkit: A grammar-based framework for urban visual analytics. *IEEE Trans. Vis. Comput. Graph.*, 30(1):1402–1412, 2024. doi: 10.1109/TVCG.2023.3326598 2
- [7] G. Moreira, M. Hosseini, C. Veiga, L. Alexandre, N. Colaninno, D. de Oliveira et al. Curio: A dataflow-based framework for collaborative urban visual analytics. *IEEE Trans. Vis. Comput. Graph.*, 2024. doi: 10.1109/TVCG.2024.3456351 1, 2
- [8] A. Satyanarayan, D. Moritz, K. Wongsuphasawat, and J. Heer. Vega-Lite: A grammar of interactive graphics. *IEEE Trans. Vis. Comput. Graph.*, 23(1):341–350, 2017. doi: 10.1109/TVCG.2016.2599030 2
- [9] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi et al. Transformers: State-of-the-art natural language processing. In *Proc. EMNLP: System Demonstrations*, pp. 38–45, 2020. doi: 10.18653/v1/2020.emnlp-demos.6 1, 2
- [10] E. Xie, W. Wang, Z. Yu, A. Anandkumar, J. M. Alvarez, and P. Luo. SegFormer: Simple and efficient design for semantic segmentation with transformers. In *Proc. NeurIPS*, 2021. 2